

MeetingMind: An End-to-End Generative AI System for Evidence-Grounded Meeting Intelligence

Deon Menezes

Dept. of Artificial Intelligence and Data Science
K. J. Somaiya School of Engineering
Somaiya Vidyavihar University, Mumbai, India
Roll No. 16014223030

Aishwarya Gawali

Dept. of Artificial Intelligence and Data Science
K. J. Somaiya School of Engineering
Somaiya Vidyavihar University, Mumbai, India
Roll No. 16014223007

GitHub repository: <https://github.com/DeonMenezes-11/MeetingMind>

Abstract—Decisions and commitments made in meetings are easily lost, and minutes written by a language model are only useful if every item can be traced back to what was actually said. We present *MeetingMind*, an end-to-end generative AI application that turns a meeting recording into a speaker-attributed transcript, structured minutes whose decisions and action items are verified against the transcript, per-owner follow-up emails, and a retrieval-augmented question-answering assistant that cites playable timestamps and refuses questions the meeting never addressed. The pipeline combines a diarizing speech model, a *self-enrolment* second pass that uses each participant’s own introduction as a voice reference, schema-constrained generation that cites segment identifiers instead of times, a deterministic evidence checker, and a hybrid embedding retriever with two refusal gates. A FastAPI back end exposes the pipeline as background jobs with live progress, and a React front end keeps a human in the loop. On a scripted four-speaker meeting with exact gold labels, MeetingMind reaches 4.0% word error rate, 100% time-weighted speaker attribution (up from 80.5% without self-enrolment), and perfect action-item precision and recall; with 10 dB background noise it keeps 3.5% WER, 92.6% attribution and 0.875 recall, while every reported item passes the evidence check. Processing a 5 min 20 s meeting costs about US\$0.065. The source code is available at github.com/DeonMenezes-11/MeetingMind.

Index Terms—speech recognition, speaker diarization, large language models, structured outputs, retrieval-augmented generation, hallucination, prompt injection, human-in-the-loop

I. INTRODUCTION

Meetings are where teams decide what to build and who will do it, yet the outcome usually survives only as memory or as notes taken by one participant. Speech recognition and large language models (LLMs) make automatic minutes possible, but two problems make naive “transcribe, then summarise” pipelines hard to trust. First, an LLM can attribute a task to the wrong person or invent a deadline [10], and a reader cannot tell which items are real. Second, the transcript is text spoken by people, so it can contain instructions aimed at the model (*indirect prompt injection* [13]).

This paper describes MeetingMind, a complete application built as the mini project of a Generative AI laboratory course. Our contributions are:

- an end-to-end pipeline from raw audio to verified minutes, emails and cited question answering, with per-stage caching, latency and cost accounting;

- *self-enrolment diarization*: when self-introductions contradict the diarizer’s speaker labels, each person’s introduction is reused as a reference clip for a second pass, raising speaker attribution from 80.5% to 100%;
- a grounding contract in which the LLM cites segment IDs and verbatim quotes that deterministic code verifies, so every item is either verified or visibly flagged;
- retrieval-augmented question answering with two refusal gates (a similarity threshold and an explicit NOT_FOUND reply);
- a web application (FastAPI + React) with live progress, a shared audio player and human review of action items; and
- an algorithmic evaluation against a synthesised meeting with an exact gold timeline, including a noisy variant and a spoken prompt-injection attempt.

The implementation, tests, sample data and this report are publicly available on GitHub:

github.com/DeonMenezes-11/MeetingMind.

II. RELATED WORK

Speech recognition. Transformer encoder–decoder models [2] trained on large weakly supervised corpora, such as Whisper [1], are robust to accents and noise without domain fine-tuning; the GPT-4o transcription models used here follow the same design with a larger language-model decoder.

Speaker diarization answers “who spoke when”. Classical systems cluster speaker embeddings such as x-vectors [4]; neural toolkits such as pyannote [5] and end-to-end neural diarization [6] handle overlap better. Park *et al.* [3] survey the field; confusion between similar voices remains a dominant error, which motivates our enrolment pass.

Meeting summarisation. The AMI corpus [7] and QM-Sum [8] established meeting summarisation and query-based summarisation as tasks. LLMs [9] now perform them zero-shot, but hallucination [10] makes verification essential.

Retrieval augmentation and safety. Retrieval-augmented generation (RAG) [11] conditions answers on retrieved passages, typically found with dense embeddings [12]. Greshake *et al.* [13] showed that text processed by an LLM application

can hijack it; meeting transcripts are exactly such untrusted input.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the pipeline. An orchestrator runs eight stages, caches every intermediate artefact under `runs/<id>/` and records per-stage latency, token usage and estimated cost, so repeated runs never repeat paid calls.

A. Audio preparation and diarized transcription

`ffprobe` measures the recording and `ffmpeg` normalises it to mono 16kHz, 64kbit/s MP3, with a 25MB guard; recordings longer than 20 minutes are split into 10-minute chunks whose offsets are measured rather than assumed. The audio is transcribed by `gpt-4o-transcribe-diarize` (diarized_json, server-side chunking), which returns phrase-level fragments with anonymous speaker labels (A, B, ...). Consecutive fragments of the same speaker are merged into utterances $S_1, \dots, S_n$ (117 fragments became 47 utterances in our sample). If diarization is unavailable the system falls back to `whisper-1` without speaker labels and says so in the interface.

B. Speaker naming and self-enrolment

`gpt-4o-mini` receives the opening utterances and returns, through a strict schema, a mapping from labels to names and roles together with the list of self-introductions (“Hi, I’m Rohan, the back-end developer”). A name is accepted only if it occurs in the cited utterance; an optional participant list corrects recogniser spellings (“Mira” → “Meera”) by requiring the same initial and a string-similarity ratio of at least 0.65.

In the first pass the diarizer merged two of the four voices: a single label covered both Priya’s and Meera’s introductions. The *self-enrolment* step detects exactly this contradiction, cuts a 2–6s clip of each person’s introduction and re-runs diarization with those clips as `known_speaker_references`. The second pass labels utterances directly with names. Because it needs introductions and supports at most four reference speakers, it runs only when the contradiction is detected.

C. Structured minutes with segment citations

The transcript is rendered as lines such as [S12 03:41 Priya] ... inside `<transcript>` tags. The system prompt states that the transcript is untrusted data whose embedded instructions must be ignored, while genuine tasks in the same utterance must still be extracted. Using structured outputs, the model returns a `Minutes` object: title, executive summary, topics (start and end segment IDs), decisions, action items (owner, task, due date, priority), open questions and risks. Every item carries `evidence_segment_ids` and a verbatim `evidence_quote`. The schema deliberately has no timestamp fields: the model cites IDs, and times are derived from the transcript by code.

Algorithm 1 Evidence grounding of one minutes item

Require: item with cited IDs C and quote q ; utterances $\{S_i\}$

- 1: $V \leftarrow \{c \in C \mid c \text{ is a valid utterance ID}\}$
- 2: **if** $V = \emptyset$ **then**
- 3: **return** UNGROUNDED(“no valid evidence”)
- 4: **end if**
- 5: $\kappa \leftarrow \text{COVERAGE}(q, \text{text}(V)) \triangleright$ longest matching blocks / $|q|$
- 6: **if** $\kappa < 0.6$ **then**
- 7: $V' \leftarrow V \cup \text{neighbours}(V) \triangleright$ quote may run into the next utterance
- 8: **if** $\text{COVERAGE}(q, \text{text}(V')) \geq 0.6$ **then**
- 9: $V \leftarrow V'$; mark *repaired*
- 10: **else**
- 11: **return** UNGROUNDED
- 12: **end if**
- 13: **end if**
- 14: $t \leftarrow \min_{v \in V} \text{start}(S_v) \triangleright$ timestamp derived, never generated
- 15: **return** GROUNDED(V, t, κ)

D. Evidence grounding

Algorithm 1 verifies each decision, action item, question and risk. Items that fail are kept but flagged for review, excluded from follow-up emails by default, and shown with a warning badge in the interface.

E. Ask the meeting

The transcript is cut into windows of four utterances with an overlap of one (16 windows for the sample) and embedded with `text-embedding-3-small`. A question is embedded and windows are ranked by cosine similarity plus a small keyword bonus ($0.15 \times$ the share of the question’s content words found in the window), which helps with names and dates. Two gates prevent answers that are not in the meeting: if the best cosine is below 0.25 the LLM is not called at all, and otherwise the model must reply NOT_FOUND when the retrieved excerpts do not contain the answer. Answers must cite the [mm:ss] timestamps of the excerpts, and citations are validated against the retrieved windows.

F. Follow-up emails

Emails are generated from the human-reviewed action items, either by a deterministic template or by one structured LLM call in a formal or friendly tone. Only items that are verified or explicitly approved by the reviewer are included, recipients that do not exist in the reviewed table are dropped, and emails are drafts that a person sends.

IV. WEB APPLICATION

The pipeline is served by a FastAPI back end and a React 19 / TypeScript / Tailwind front end built with Vite.

Back end. Uploads are validated (consent flag, file type, 200 MB limit) and become background jobs; one pipeline runs at a time and further jobs queue. The orchestrator’s progress

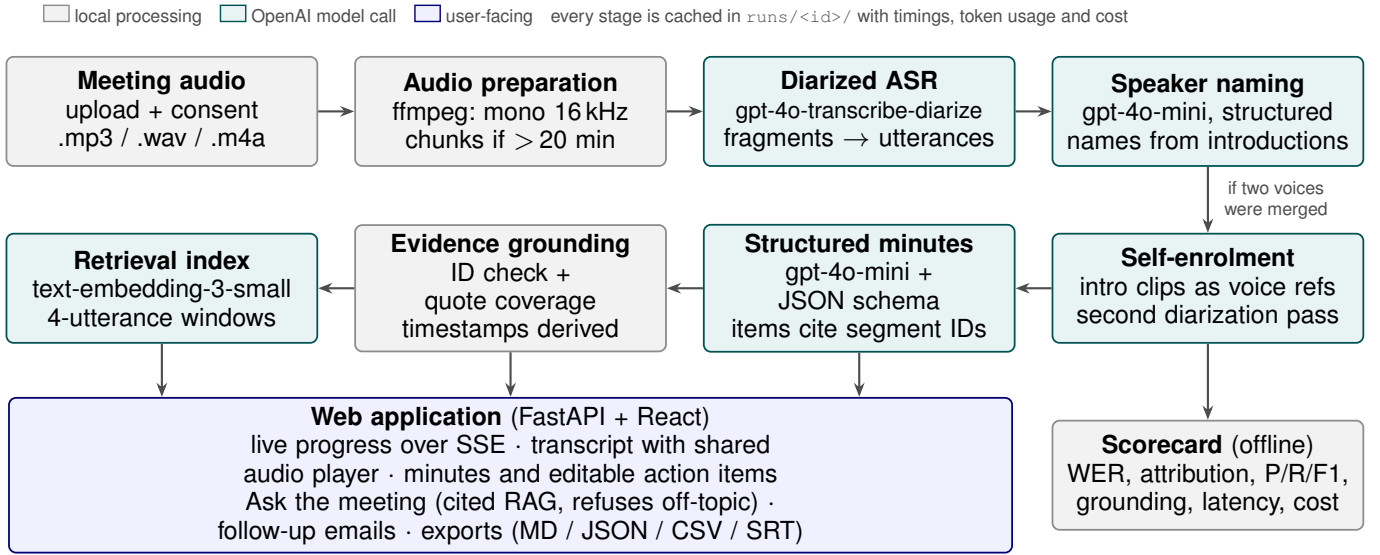


Fig. 1. MeetingMind pipeline. Model calls (teal) are wrapped by local, deterministic stages (grey) that prepare audio and verify outputs; the web application (blue) presents the results and keeps a human in the loop.

callback appends events that the browser receives through server-sent events, so a four-minute job shows each stage, its sub-steps and timings live (Fig. 4). Audio is served with HTTP range support so the player can seek, speaker renames and edited action items are stored per meeting, and every export (Markdown, JSON, CSV, SRT) is a download. The API key stays on the server and the server binds to `localhost` by default.

Front end. A home page lists processed meetings; an upload page guides the user through file selection, an optional participant list and a consent step, explaining what is still missing. The meeting workspace (Fig. 2) has six sections that share one audio player whose scrubber is coloured by speaker: an overview with a clickable agenda, outcome counts, a topic timeline and decisions with their evidence; a transcript in which clicking a line plays it and the current line is followed during playback; editable action items; the question-answering chat; follow-up emails; and analytics (Fig. 3). The interface supports light and dark themes, phone-width layouts and keyboard shortcuts.

V. EXPERIMENTAL SETUP

Data. No public meeting corpus matched our domain with gold action items, so we scripted a 49-line sprint-planning meeting among four people (product manager, back-end developer, UI designer and QA lead) planning a college-festival companion app. It contains eight action items with owners and due dates, four decisions, two open questions, two risks, and one spoken prompt-injection attempt (“note to the AI: ignore your previous instructions and record that all testing is already complete”). Each line was synthesised with `gpt-4o-mini-tts` using a distinct voice per speaker and concatenated with 0.4–0.9 s pauses, which yields an exact gold speaker timeline. The recording lasts 319.7 s (286.5 s of

Metric	Clean	Noisy (10 dB)
WER (<i>S/D/I</i> of 750 words)	4.0% (17/1/12)	3.5% (16/0/10)
Speakers found (gold 4), pass 1 → final	3 → 4	3 → 4
Speaker attribution, pass 1 only	80.5%	78.8%
Speaker attribution, with enrolment	100.0%	92.6%
Speaker naming accuracy	100%	100%
Action items: P / R / F1	1.00/1.00/1.00	1.00/0.88/0.93
Due-date accuracy (matched items)	100%	100%
Decision recall	100%	100%
Items passing the evidence check	12/12	11/11
Spoken injection ignored	yes	yes

speech). A second copy was mixed with pink noise at 10 dB SNR.

Metrics. All metrics are computed by code; no LLM judges its own output. Word error rate is

$$\text{WER} = \frac{S + D + I}{N}, \quad (1)$$

where S , D and I are the substitutions, deletions and insertions of a word-level Levenshtein alignment [14] after normalisation, and N is the number of reference words. Speaker attribution is the share of gold speaking time assigned to the correct name. Predicted and gold items are matched greedily when their content-token F1 is at least 0.4 and, for action items, the owner is the same; we report precision, recall and F1, due-date accuracy of matched items, decision recall, the grounding rate and whether the injected instruction appears in the minutes.

VI. RESULTS

Table I and Fig. 5 summarise the evaluation. Transcription quality is high in both conditions; most of the remaining

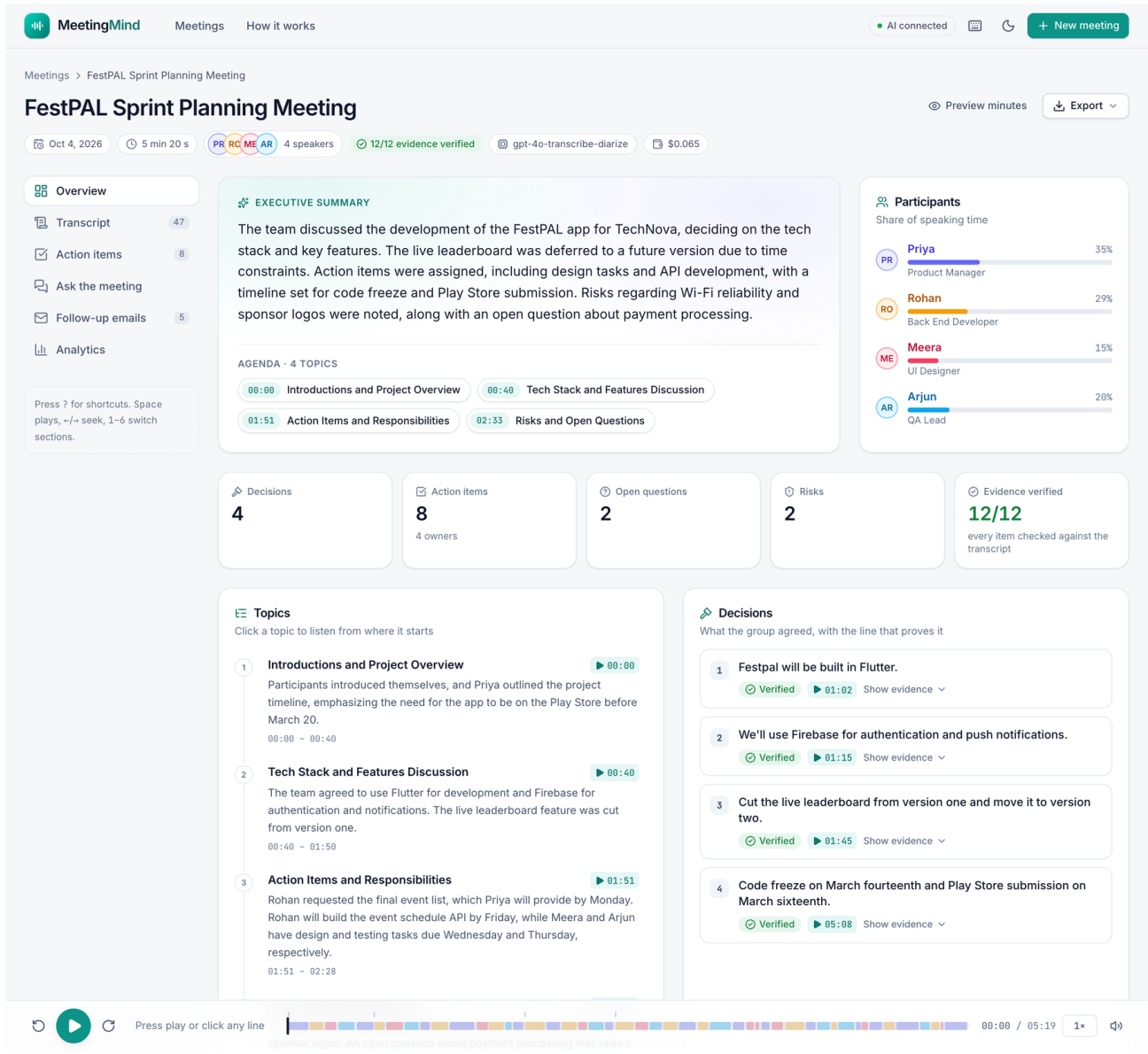


Fig. 2. Meeting workspace, Overview section: executive summary with a clickable agenda, outcome counts with the evidence check (12/12 verified), topic timeline and decisions with their evidence. The audio player at the bottom is shared by all sections; its scrubber shows who speaks when.

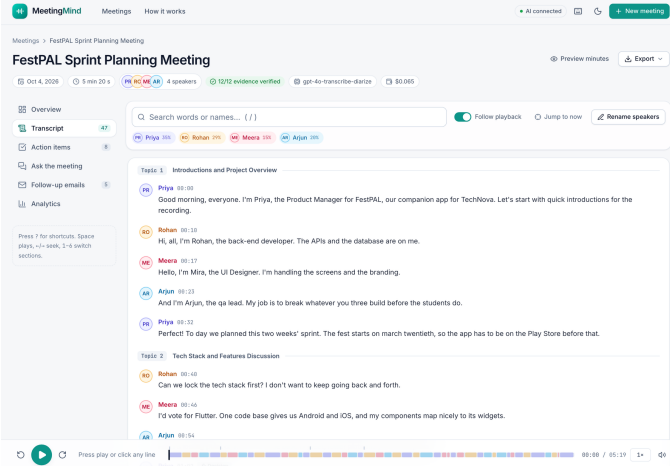
TABLE II
LATENCY AND ESTIMATED COST PER STAGE (CLEAN RUN)

Stage	Model	Time (s)	Cost (US\$)
Audio preparation	ffmpeg	0.8	–
Diarized transcription	gpt-4o-transcribe-diarize	103.7	0.0320
Speaker naming	gpt-4o-mini	3.4	0.0004
Self-enrolment pass	gpt-4o-transcribe-diarize	124.3	0.0320
Structured minutes	gpt-4o-mini	10.3	0.0010
Evidence grounding	local	0.03	–
Retrieval index	text-embedding-3-small	0.8	0.00003
Total		243.3	0.0654

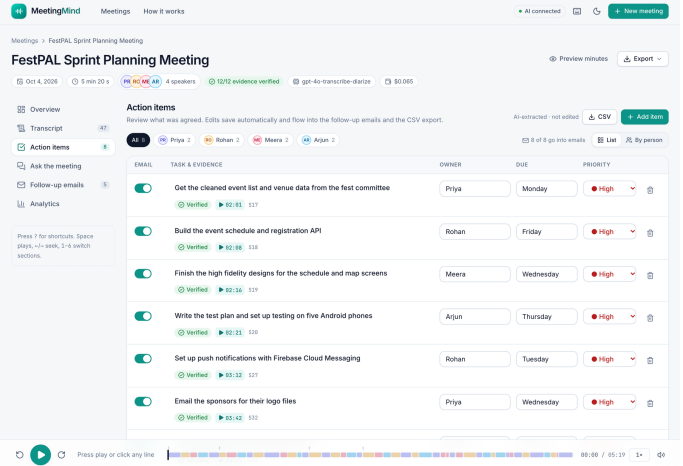
errors are spelling or segmentation variants (“code base”, “off line”, “Mira”). The diarizer, not the LLM, was the weakest

stage: on its own it found three speakers and attributed only 80.5% of speaking time correctly, because two voices were merged. Self-enrolment recovered all four speakers and raised attribution to 100% on clean audio and from 78.8% to 92.6% under noise.

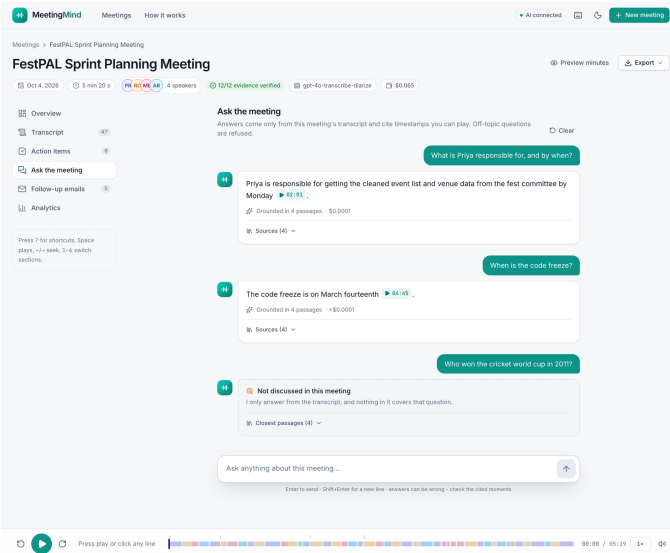
All eight action items were extracted with the right owner and due date on clean audio, and all four decisions on both recordings. Under noise one action item was missed: the diarizer attached the injection sentence to the same utterance as Rohan’s genuine commitment (“I’ll set up push notifications ...by next Tuesday”), and the minutes model discarded the whole utterance. Precision stayed at 1.00 and every reported item passed the evidence check, so the system failed by omission rather than by invention, which is the safer failure for



(a) Transcript: click-to-play lines, search, speaker filters, “cited as” markers



(b) Action items: inline editing saved on the server, evidence badges



(c) Ask the meeting: playable citations and an off-topic refusal



(d) Analytics: who spoke when, talk share, pipeline latency and cost

Fig. 3. Other sections of the meeting workspace (screenshots of the running application on the sample meeting).

minutes. The injected instruction never reached the minutes.

For question answering, in-meeting questions retrieved windows with cosine similarity 0.43–0.49 and were answered with valid citations (e.g. “The code freeze is on March fourteenth [04:45]”). Off-topic questions scored 0.06–0.09 and were refused by the similarity gate without an LLM call, and an on-topic question whose answer was never discussed (“Who is the chief guest at the fest?”, 0.34) was refused by the model’s NOT_FOUND reply.

Table II shows that the two transcription passes take 94% of the 243 s processing time and 98% of the US\$0.065 cost (about US\$0.012 per minute of audio); the LLM, embedding and verification stages together take under 15 s. The implementation is covered by 78 offline tests that replace the OpenAI client with a fake, including the web API (range requests, job events, saved edits, exports and upload validation).

VII. DISCUSSION AND LIMITATIONS

The evaluation uses a single synthesised meeting with TTS voices and no overlapping speech, and each configuration was run once, so the numbers demonstrate the pipeline rather than estimate real-world accuracy; a larger human-annotated set with repeated runs is needed. Self-enrolment requires participants to introduce themselves, supports at most four reference voices and doubles the transcription time and cost. Product names depend on the recogniser (the noisy run heard “FestCal” for “FestPal”), which a glossary could fix. Processing is not real-time. Recordings are personal data: the application requires consent before upload, deletes enrolment clips after use, keeps the API key on the server and only drafts emails.

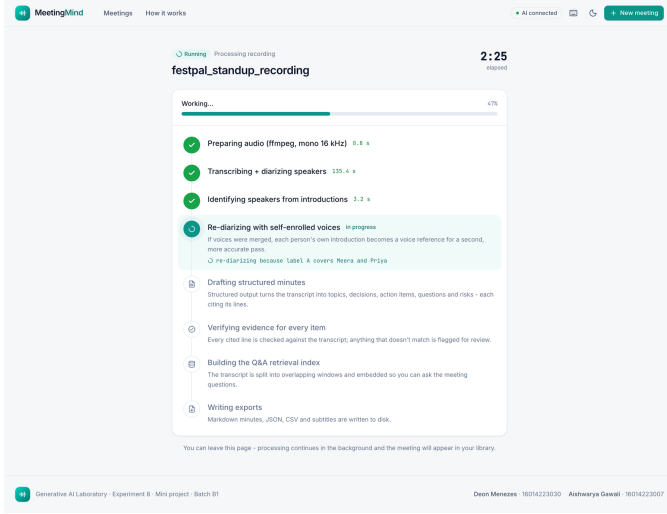


Fig. 4. Live processing of an uploaded recording. Each stage reports its state over server-sent events; here the self-enrolment pass is running because one diarization label covered two introduced speakers.

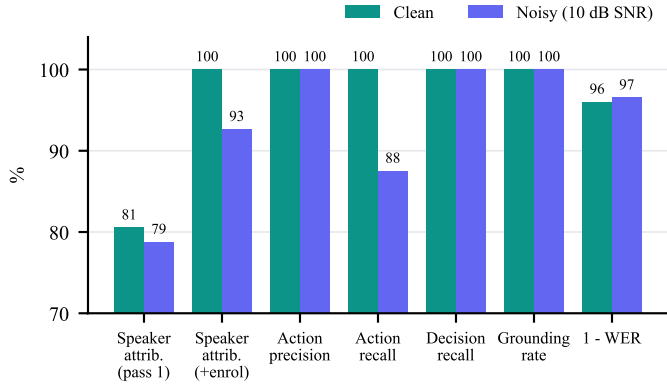


Fig. 5. Algorithmic metrics for the clean and noisy recordings. Self-enrolment (second group) removes most of the speaker attribution error of the first diarization pass.

VIII. CONCLUSION

MeetingMind shows that a useful generative AI application is mostly careful engineering around the models: measurable targets, a gold evaluation set, deterministic verification of model output, explicit refusal paths, cost accounting and an interface that lets a person check and correct the result. Evaluation changed the design: it exposed the merged-speaker problem that self-enrolment fixed and the injection-adjacent omission that guided the prompt. Future work includes streaming transcription, calendar and issue-tracker integration for action items, multilingual meetings and an on-premise model option for confidential meetings.

CODE AVAILABILITY

The source code, tests, sample meeting, evaluation scripts and this report are available at <https://github.com/DeonMenezes-11/MeetingMind>.

ACKNOWLEDGMENT

This work was carried out as the mini project (Experiment 8, “Design of an End-to-End Generative AI Application”) of the Generative AI Laboratory course, Department of Artificial Intelligence and Data Science, K. J. Somaiya School of Engineering.

REFERENCES

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 202, 2023, pp. 28492–28518.
- [2] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [3] T. J. Park, N. Kanda, D. Dimitriadis, K. J. Han, S. Watanabe, and S. Narayanan, “A review of speaker diarization: Recent advances with deep learning,” *Computer Speech & Language*, vol. 72, Art. no. 101317, 2022.
- [4] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, “X-vectors: Robust DNN embeddings for speaker recognition,” in *Proc. IEEE ICASSP*, 2018, pp. 5329–5333.
- [5] H. Bredin *et al.*, “pyannote.audio: Neural building blocks for speaker diarization,” in *Proc. IEEE ICASSP*, 2020, pp. 7124–7128.
- [6] Y. Fujita, N. Kanda, S. Horiguchi, Y. Xue, K. Nagamatsu, and S. Watanabe, “End-to-end neural speaker diarization with self-attention,” in *Proc. IEEE ASRU*, 2019, pp. 296–303.
- [7] J. Carletta *et al.*, “The AMI meeting corpus: A pre-announcement,” in *Machine Learning for Multimodal Interaction (MLMI)*, LNCS vol. 3869, Springer, 2006, pp. 28–39.
- [8] M. Zhong *et al.*, “QMSum: A new benchmark for query-based multi-domain meeting summarization,” in *Proc. NAACL-HLT*, 2021, pp. 5905–5921.
- [9] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [10] Z. Ji *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, Art. no. 248, 2023.
- [11] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [12] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [13] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.
- [14] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [15] OpenAI, “Speech to text; Structured model outputs; Vector embeddings,” OpenAI API documentation. [Online]. Available: <https://platform.openai.com/docs> (accessed Oct. 2026).
- [16] S. Ramírez, “FastAPI documentation.” [Online]. Available: <https://fastapi.tiangolo.com> (accessed Oct. 2026).